Accessing Your Applications from Outside

Ingress & Egress — Module 6 of 13

Enterprise EKS Platform

📋 Module Agenda

- Why External Access Matters
- NGINX Ingress Controller & IngressClass
- Ingress Resources: Path-Based & Host-Based Routing
- Annotations, TLS Termination & cert-manager
- Gateway API: GatewayClass, Gateway, HTTPRoute
- Traffic Splitting and Canary Deployments
- Enterprise Integration & Egress Controls

Why External Access Matters

Getting Traffic Into Your Cluster

🔌 External Access Options

Method	Layer	Use Case	Limitation
`NodePort`	L4	Dev/testing	Port range 30000-32767, exposed on all nodes
`LoadBalancer`	L4	Single service exposure	One LB per service ($$$), no path routing
Ingress	L7	HTTP/HTTPS routing	HTTP-only, limited extensibility
Gateway API	L4-L7	Advanced routing	Newer, still maturing ecosystem

NGINX Ingress Controller

Architecture and IngressClass

🏗️ NGINX Ingress Architecture

How It Works

1. NGINX Ingress Controller pod runs NGINX reverse proxy
2. Exposed via a `LoadBalancer` Service (single AWS NLB/ALB)
3. Watches for Ingress resources across the cluster
4. Dynamically generates NGINX configuration from Ingress rules
5. Reloads NGINX when Ingress resources change

🔧 IngressClass Configuration

What IngressClass Does: Decouples the Ingress resource from the controller. Setting `is-default-class` means any Ingress without an explicit `ingressClassName` is handled by that controller.

Recommended Pattern: Run separate IngressClasses for external (internet-facing) and internal (VPC-only) traffic, each with its own controller and load balancer.

Ingress Resources

Path-Based and Host-Based Routing

📌 Path Types Explained

pathType	Matching Behavior	Example
`Exact`	Matches URL path exactly	`/api` matches `/api` only, not `/api/v1`
`Prefix`	Matches path prefix by `/` segments	`/api` matches `/api`, `/api/v1`, `/api/v1/users`
`ImplementationSpecific`	Depends on IngressClass controller	NGINX supports regex: `/api(/|$)(.*)`

🔧 Ingress Annotations

Customizing NGINX Behavior

🔒 TLS Termination

Securing Ingress with HTTPS

🔶 Gateway API

The Evolution of Kubernetes Ingress

Ingress vs Gateway API

Aspect	Ingress	Gateway API
Extensibility	Annotations (untyped strings)	Typed, structured API fields
Role model	Single resource, one owner	Role-oriented (infra, cluster, app)
Protocols	HTTP/HTTPS only	HTTP, TCP, UDP, gRPC, TLS
Traffic splitting	Not native	Built-in weight-based routing
Header matching	Not supported	Native header/query matching
Portability	Vendor-specific annotations	Portable across implementations
Status	Stable, feature-frozen	GA (HTTPRoute), evolving

🏗️ Gateway API Resource Model

🛠️ GatewayClass

Role: Infrastructure Provider

- Defines controller implementation (cluster-scoped)

🔶 Gateway

Role: Cluster Operator

- Configures listeners, TLS, and provisions LB

📐 HTTPRoute

Role: Application Developer

- Defines routing rules; attaches to Gateway

Role Separation: Infrastructure teams manage GatewayClass and Gateway. Application teams create HTTPRoutes. RBAC applies cleanly at each level.

⚖️ Traffic Splitting

Canary Deployments with HTTPRoute Weights

Enterprise Integration

Envoy Gateway + External DNS + cert-manager

🏙️ Enterprise EKS Ingress Stack

🔶 Envoy Gateway

- Gateway API + Envoy data plane
- Advanced L7: gRPC + HTTP

🌐 External DNS

- Watches Gateway/Ingress
- Auto-creates Route53 records

🔒 cert-manager

- Automated cert lifecycle
- Internal CA, ACME, or Gateway API

Workflow: HTTPRoute → Envoy configures proxy → External DNS creates Route53 record → cert-manager provisions TLS → App is live.

Egress Controls

Managing Outbound Traffic

⬅️ END Why Egress Controls Matter

🛡️ Security Concerns

- Compromised pods phoning home or exfiltrating data
- Supply chain attacks reaching external registries
- Compliance requirements (PCI-DSS, SOC2)

🔧 Control Mechanisms

- **NetworkPolicy egress rules** (Module 8)
- **Egress gateways** (service mesh) or **NAT gateways**
- **Proxy servers** for HTTP traffic

🔌 Egress Patterns on EKS

NAT Gateway (Default EKS)

- Pods egress via the VPC NAT gateway with a fixed Elastic IP
- All egress shares the NAT GW IP — useful for IP allowlisting
- No application-level control

Custom CA for External Endpoints

- Mount a CA bundle Secret into pods for mTLS with external APIs
- Trust private CAs without rebuilding container images

👨‍💻 Lab 6: Ingress and Gateway API

Hands-On Exercise

- Verify the NGINX Ingress Controller and IngressClass
- Create host-based and path-based Ingress resources
- Configure TLS termination with a self-signed certificate
- Test Ingress annotations: rate limiting, CORS, rewrite
- *Optional:* Deploy Gateway API resources with HTTPRoute traffic splitting
- *Optional:* Configure egress NetworkPolicies to control outbound traffic

> **Duration:** ~30-40 minutes

🎯 Key Takeaways

1. Consolidate with Ingress/Gateway — Avoid one LoadBalancer per service. Route through a shared entry point.

2. Gateway API is the future — Typed resources, role separation, and native traffic management replace annotation-driven Ingress.

3. Automate TLS — Use cert-manager for certificate lifecycle. Avoid managing certificates manually.

4. Control egress too — Ingress is only half the story. Egress controls are critical for security and compliance.

? Questions & Discussion

Module 6: Accessing Your Applications from Outside — Ingress & Egress

Up Next: Module 7 — RBAC & Security